authentication flow but delegates the actual authentication step to Keycloak using the OpenID Connect protocol.

Once a user authenticates successfully with Keycloak, Edge Service receives an ID Token from Keycloak with information about the authentication event and initiates an authenticated session with the user's browser. At the same time, Keycloak also issues an *Access Token*, which is used to grant Edge Service access to downstream applications on behalf of the user as per OAuth2.

OAuth2 is an authorization framework that enables an application (called a *Client*) to obtain limited access to a protected resource provided by another application (called a *Resource Server*) on behalf of a user. When a user authenticates with Edge Service and asks to access their book orders, OAuth2 provides a solution for Edge Service to retrieve orders from Order Service on behalf of that user. This solution relies on a trusted party (called an *Authorization Server*), which issues an *Access Token* to Edge Service and grants access to the user's book orders from Order Service.

You might recognize some of these roles from the OIDC authentication flow we adopted in the previous chapter. As anticipated, OIDC is an identity layer built on top of OAuth2 and relies on the same basic concepts:

- *Authorization Server*—The entity responsible for authenticating users and issuing, refreshing, and revoking Access Tokens. In Polar Bookshop, this is Keycloak.
- *User*—Also called the *Resource Owner*, this is the human logging in with the Authorization Server to get authenticated access to the Client application. It's also the human or service granting a Client access to the protected resources provided by a Resource Server. In Polar Bookshop, it's either a customer or an employee.